

process.

2. Hold and Wait

- To prevent this condition processes must be prevented from holding one or more resources while simultaneously waiting for one or more others. There are several possibilities for this:
 - Require that all processes request all resources at one time. This can be wasteful of system resources if a process needs one resource early in its execution and doesn't need some other resource until much later.
 - Require that processes holding resources must release them before requesting new resources, and then re-acquire the released resources along with the new ones in a single new request. This can be a problem if a process has partially completed an operation using a resource and then fails to get it re-allocated after releasing it.
 - Either of the methods described above can lead to starvation if a process requires one or more popular resources.

3. No Pre-emption

- Pre-emption of process resource allocations can prevent this condition of deadlocks, when it is possible.
 - One approach is that if a process is forced to wait when requesting a new resource, then all other resources previously held by this process are implicitly released, (pre-empted), forcing this process to re-acquire the old resources along with the new resources in a single request, similar to the previous discussion.
 - Another approach is that when a resource is requested and not available, then the system looks to see what other processes currently have those resources *and* are themselves blocked waiting for some other resource. If such a process is found, then some of their resources may get pre-empted and added to the list of resources for which the process is waiting.
 - Either of these approaches may be applicable for resources whose states are easily saved and restored, such as registers and memory, but are generally not applicable to other devices such as printers and tape drives.

4. Circular Wait